

Mini ERP System

Inventory and Sales Management System

Technology Stack

Frontend: React

Backend: Flask (REST API)

Database: PostgreSQL

Containerization: Docker Compose

UI Framework: TailwindCSS + DaisyUI

1. Introduction

Enterprise Resource Planning (ERP) systems help businesses manage important operational processes such as inventory management, product tracking, and sales transactions.

This project proposes the development of a **Mini ERP System** designed for small businesses. The system will allow users to manage products, suppliers, and sales while automatically maintaining inventory records.

The system will follow a **modern full-stack architecture** using **React for the frontend** and **Flask for the backend API**, with **PostgreSQL** used for persistent data storage.

The project will emphasize **clean architecture, modular structure, and good coding practices**.

2. Objectives

Main Objective

To develop a **simple ERP web application** that manages products, suppliers, sales, and inventory.

Specific Objectives

- Build a RESTful API using Flask
- Develop a responsive UI using React
- Manage product inventory

- Track suppliers
 - Record sales transactions
 - Automatically update stock levels
 - Implement role-based access
-

3. System Architecture

React Frontend

|

| REST API

|

Flask Backend

|

SQLAlchemy ORM

|

PostgreSQL Database

The system will be containerized using **Docker Compose**.

4. User Roles

The system will have two roles.

1. Admin

Admin manages the entire system.

Permissions:

- Manage users
- Create, edit, delete products
- Manage suppliers
- View all sales

- View dashboard statistics
 - Manage inventory
-

2. Staff

Staff handles daily sales operations.

Permissions:

- View products
- Record sales
- View suppliers
- View inventory

Staff cannot delete products or manage users.

5. Core Modules

Authentication Module

Handles login and role verification.

Features:

- Login
 - Logout
 - Token authentication
-

Product Management

Features:

- Add product
- Edit product
- Delete product
- View product list

Supplier Management

Features:

- Add supplier
- Update supplier
- View supplier list

Sales Management

Features:

- Record sales
- Select product
- Enter quantity
- Calculate total price

Inventory Management

Features:

- Track product stock
- Reduce stock after sales
- Display low stock alerts

Dashboard

Displays summary statistics:

- Total products
 - Total sales
 - Low stock products
-

6. Database Schema

Users Table

Field	Type
id	Integer
username	String
password_hash	String
role	String
created_at	Timestamp

Suppliers Table

Field	Type
id	Integer
name	String
email	String
phone	String
address	Text
created_at	Timestamp

Products Table

Field	Type
id	Integer
name	String
price	Float

Field	Type
stock_quantity	Integer
supplier_id	Integer
created_at	Timestamp

Sales Table

Field	Type
id	Integer
product_id	Integer
quantity	Integer
total_price	Float
sold_by	Integer
created_at	Timestamp

7. Entity Relationships

Supplier



Products



Users

8. REST API Design

Authentication

POST /api/auth/login

Request

```
{  
  "username": "admin",  
  "password": "password"  
}
```

Response

```
{  
  "token": "jwt_token",  
  "role": "admin"  
}
```

Product APIs

GET /api/products

GET /api/products/{id}

POST /api/products

PUT /api/products/{id}

DELETE /api/products/{id}

Supplier APIs

GET /api/suppliers

POST /api/suppliers

PUT /api/suppliers/{id}

DELETE /api/suppliers/{id}

Sales APIs

POST /api/sales

Request

```
{  
  "product_id": 1,  
  "quantity": 3  
}
```

The backend will:

- calculate total price
 - reduce stock automatically
-

GET /api/sales

Dashboard API

GET /api/dashboard/stats

Example Response

```
{  
  "total_products": 30,  
  "total_sales_today": 15,  
  "low_stock_products": 4  
}
```

9. Project Folder Structure

mini-erp-system

docker-compose.yml

README.md

backend

|

|— app

|

|— __init__.py

|

|— config.py

|

|— extensions.py

|

|

|— models

|

|— user.py

|

|— product.py

|

|— supplier.py

|

|— sale.py

|

|

|— routes

|

|— auth_routes.py

|

|— product_routes.py

|

|— supplier_routes.py

|

|— sales_routes.py

|

```
| |─ services
| | |─ inventory_service.py
| | |─ sales_service.py
|
| |─ utils
| | |─ auth_utils.py
|
|─ migrations
|─ tests
|─ requirements.txt
└─ run.py
```

frontend

```
|
|─ public
|
└─ src
    |
    |─ api
    | |─ apiClient.js
    |
    |─ components
    | |─ Navbar.jsx
    | |─ Sidebar.jsx
```

```
|
|
|└─ pages
|
|  |└─ Dashboard.jsx
|
|  |└─ Products.jsx
|
|  |└─ Suppliers.jsx
|
|  |└─ Sales.jsx
|
|  └─ Login.jsx
|
|
|└─ context
|
|  └─ AuthContext.jsx
|
|
|└─ App.jsx
└─ main.jsx
```

10. Libraries Used

Backend

Flask
Flask-SQLAlchemy
Flask-Migrate
Flask-JWT-Extended
Flask-CORS
psycpg2-binary
python-dotenv

Frontend

React
Axios
React Router DOM

TailwindCSS
DaisyUI
Chart.js
React Chartjs 2

Database

PostgreSQL

DevOps

Docker
Docker Compose

11. Development Phases

Phase 1 – Project Setup

- Setup Git repository
 - Create React project
 - Create Flask project
 - Setup PostgreSQL
 - Configure Docker Compose
-

Phase 2 – Authentication

- Create User model
 - Implement login API
 - JWT authentication
 - React login page
-

Phase 3 – Product Module

- Product database model
 - Product CRUD APIs
 - React product management pages
-

Phase 4 – Supplier Module

- Supplier model
 - CRUD APIs
 - Supplier UI pages
-

Phase 5 – Sales Module

- Sales model
 - Record sales API
 - Stock reduction logic
-

Phase 6 – Dashboard

- Statistics API
 - Dashboard charts
-

Phase 7 – Testing and Finalization

- API testing
 - Error handling
 - Code refactoring
 - Documentation
-

12. Coding Best Practices

Backend

- Use Flask Blueprints
- Separate routes, models, and services
- Use environment variables
- Use SQLAlchemy ORM
- Implement JWT authentication

Frontend

- Use reusable components
- Use API service layer
- Use React Context for authentication
- Organize pages clearly

General

- Use Git version control
- Maintain consistent project structure
- Write clear commit messages
- Document APIs

13. Expected Outcome

At the end of the project, the system will:

- Provide a working Mini ERP web application
- Allow management of products and suppliers
- Record sales and update inventory automatically
- Display business statistics on a dashboard
- Demonstrate modern full-stack development with React and Flask

14. Future Improvements

Possible future enhancements include:

- Purchase management module
- Advanced reporting
- Multi-warehouse inventory
- Notification system
- Analytics dashboard

END OF DOCUMENT